Unit 4: Computational Structures

Complete exercise solutions — MCQs, Short Questions, Long Questions

EXERCISE SOLUTIONS — Unit 4: Computational Structures

Multiple Choice Questions

1. The function used to add an item at the end of a list in Python is:

- a) insert()
- b) append()
- c) remove()
- d) pop()

Answer: b) append()

2. The purpose of the 'in' keyword used with a Python list is:

- a) Adds an item to the list
- b) Removes an item from the list
- c) Checks if an item exists in the list
- d) Returns the length of the list

Answer: c) Checks if an item exists in the list

3. An operation that removes an item from the top of the stack is:

- a) Push
- b) Pop
- c) Peek
- d) Add

Answer: b) Pop

4. The operation used to add an item to a queue is:

- a) Dequeue
- b) Peek
- c) Enqueue
- d) Remove

Answer: c) Enqueue

5. True statement about the height of a tree is:

- a) Number of edges from the root to the deepest node
- b) Number of nodes from the root to the deepest node
- c) Number of children of the root node
- d) Always equal to the number of nodes in the tree

studysapiens.com

Answer: a) Number of edges from the root to the deepest node

6. A scenario where a graph data structure is most suitable is:

• a) Managing a to-do list

• b) Modelling a line of customers in a store

• c) Representing connections in a social network

• d) All of the above

Answer: c) Representing connections in a social network

Short Questions

1. Explain how the insert() function works in Python lists. Provide an example.

The insert() function adds an item at a specified position in a list, shifting the existing elements to the right. It takes two arguments: the index and the value. Example: fruits = ['apple', 'banana']; fruits.insert(1, 'orange') results in ['apple', 'orange', 'banana']. Unlike append(), which always adds at the end, insert() allows adding at any position. If the index is greater than the list length, the item is added at the end.

2. Explain the potential issues which could arise when two variables reference the same list in a program. Provide an example.

In Python, lists are mutable objects. If two variables reference the same list (e.g., a = [1, 2, 3]; b = a), then b is not a copy — both variables point to the same list. Modifying the list through one variable changes it for the other: a.append(4) also changes b to [1, 2, 3, 4]. This can cause unexpected bugs. To avoid this, use a copy: b = a.copy() or b = list(a). Example of the issue: a = [1, 2]; b = a; b.append(3); print(a) gives [1, 2, 3], not [1, 2].

3. Define a stack and explain the Last-In, First-Out (LIFO) principle.

A stack is a linear data structure that follows the LIFO principle — the last item added is the first one removed. It has two main operations: push (add an item on top) and pop (remove the top item). Real-life example: a stack of books — you take the top book first. In the textbook example, pushing 'Book A' then 'Book B' and then popping removes 'Book B' first, because it was added last. Stacks are used in undo operations, function call management, and expression evaluation.

4. Differentiate between the Enqueue and Dequeue operations of a queue.

Enqueue adds an item to the back (rear) of the queue. Dequeue removes an item from the front of the queue. A queue follows the FIFO (First-In, First-Out) principle — the first item added is the first one removed, like a line at a bank. Example: if people join a bank line (enqueue) in order A, B, C, then A is served first (dequeue). Enqueue increases the queue size; dequeue decreases it.

5. Name two basic operations performed on a stack.

The two basic operations on a stack are: (1) Push — adds an item to the top of the stack. (2) Pop — removes the top item from the stack. A third common operation is Peek, which views the top item without removing it. Push and pop are the fundamental operations that define stack behaviour.

6. What is the difference between enqueue() and dequeue()?

Enqueue() adds an item to the rear (back) of the queue, while dequeue() removes an item from the front of the queue. Enqueue increases the size of the queue; dequeue decreases it. Together they implement the FIFO principle: items leave in the same order they arrived. Example: enqueue(10), enqueue(20), dequeue() returns 10 (the first item added).

Long Questions

1. Discuss the dynamic size property of lists in Python. How does this property make lists more flexible?

Unlike arrays in languages like C or Java, Python lists have dynamic size — they can grow and shrink at runtime. You can add items with append() or insert(), remove items with remove() or pop(), and the list automatically adjusts its memory. This makes lists more flexible because: (1) you do not need to know the size in advance; (2) you can build lists incrementally as data arrives; (3) you can easily modify the list during program execution. The trade-off is slightly more memory overhead compared to fixed-size arrays, but the flexibility is worth it for most applications.

2. Explain the operations on stack with a real-life example and Python code.

A stack follows LIFO. Real-life example: a stack of plates — you add plates on top and take from the top. Python implementation using a list: stack = []; stack.append(item) for push; stack.pop() for pop; stack[-1] for peek. Example code: stack = []; stack.append('Book A'); stack.append('Book B'); print(stack.pop()) prints 'Book B' (the last added). Stacks are used in browser back buttons, undo features, and function call stacks.

3. Write a simple program to implement a queue (insertion and deletion).

from collections import deque queue = deque() queue.append(10) # enqueue queue.append(20) # enqueue queue.append(30) # enqueue print('Queue:', list(queue)) print('Dequeued:', queue.popleft()) # dequeue -> 10 print('Queue after dequeue:', list(queue)) Explanation: deque.append() adds to the rear (enqueue), and deque.popleft() removes from the front (dequeue). This follows FIFO: 10 was added first, so it is removed first. Alternatively, a simple list can be used: queue.append(x) for enqueue and queue.pop(0) for dequeue.

4. Define a tree and explain its properties.

A tree is a hierarchical data structure that organises data starting from a root node, with nodes connected by edges. Each node can have child nodes, and the structure branches downward. Key properties: (1) Root — the topmost node with no parent. (2) Edge — the connection between a parent and child. (3) Leaf — a node with no children. (4) Height — the longest path from the root to the farthest leaf. (5) Balanced tree — left and right branches are nearly equal in height, ensuring efficient operations. Trees are used in file systems, search trees, and organisational charts.

5. What is a graph? Explain differences between directed and undirected graphs.

A graph is a data structure consisting of vertices (nodes) connected by edges. It represents relationships between entities. A directed graph has edges with a one-way direction — for example, a one-way road from city A to city B, or a 'follows' relationship on social media. An undirected graph has edges with no direction — the connection works both ways, like a mutual friendship or a two-way road. Graphs can also be weighted (edges have values, like distance or cost) or unweighted. They are used in maps, social networks, and route planning.

studysapiens.com